

Auditable Personal Memory for User-Adaptive Agents

Marcel Osmond

Queen Mary University of London

Singapore Management University

marcelosmond75@gmail.com

Abstract

Personal intelligence depends on memory. A personal agent that cannot retain preferences, constraints, prior failures, and evolving context cannot adapt to a user over time. Yet the same memory mechanisms that make user-adaptive agents useful also create a governance and evaluation problem: users may not know what was stored, how it was transformed, whether it was retrieved, whether it entered a shared or parametric representation, or whether deletion and correction requests have any operational effect. This paper argues that personal agent evaluation should therefore include memory auditability as a first-class evaluation dimension alongside task success, personalization quality, privacy, and safety. We synthesize recent work on learned memory design, indexed experience memory, hierarchical retrieval, control-centric memory update, personalization architectures, parametric reflective memory, shared cross-agent memory, interdependent memory benchmarks, skill libraries, and test-time memory evolution. Across these systems, memory is moving from explicit storage toward adaptive infrastructures that decide what to retain, compress, retrieve, consolidate, share, or convert into reusable capabilities. We contribute the Personal Memory Audit Protocol, a reporting and stress-testing framework for evaluating user-adaptive agents across six questions: source lineage, transformation traceability, retrieval accountability, correction and deletion effectiveness, cross-agent propagation, and user-comprehensible inspection. The protocol is intended as a lightweight layer that can be added to personalization benchmarks and deployment evaluations without requiring a new dataset.

Keywords

personal agents, memory auditability, personalization, user modeling, agent evaluation, privacy

1 Introduction

Personalized agents need memory. A travel assistant must remember accessibility constraints, budget preferences, passport deadlines, and past booking failures. A coding assistant may need to remember repository conventions and a user’s preferred debugging workflow. A tutoring agent may need to track misconceptions and learning pace. A legal or health self-help agent may need to remember facts that are sensitive, contextual, and consequential. In each case, the promise of personal intelligence is not only better response generation. It is adaptation across time.

This creates a tension that current evaluation practice does not fully capture. Personalization benchmarks tend to reward whether a system behaves better after observing user information. Privacy and safety evaluations tend to ask whether sensitive information is leaked or misused. Both questions matter, but neither is sufficient.

A user-adaptive system can improve personalization while making memory behavior less inspectable. It can store a user’s preference in a readable database, decompose it into components, consolidate it into a profile, retrieve it through a learned controller, share an abstraction across agents, or convert interaction-derived patterns into parametric or skill-like state. These choices affect whether a user, evaluator, or deployer can answer basic questions: what did the agent remember, why did it remember it, where did it go, and can it be corrected or removed?

This paper argues that personal agent evaluation should include *memory auditability*: the capacity to inspect, trace, test, and contest how a user-adaptive agent stores and uses user-derived information. Memory auditability is not identical to privacy. Privacy asks whether information is collected, protected, and disclosed appropriately. Auditability asks whether memory operations remain reconstructable enough for users and evaluators to understand how personal information changes agent behavior. A private but opaque memory system may still leave users unable to verify what the agent knows about them or whether correction and deletion requests work.

The need for this evaluation layer follows from the direction of recent agent memory research. Retrieval-augmented generation showed the value of combining parametric and non-parametric memory, with non-parametric stores offering more direct opportunities for inspection and updating [5]. Newer agent-memory systems are more dynamic. ALMA meta-learns executable memory designs rather than relying on fixed human-crafted mechanisms [11]. Memex trains agents to decide what to summarize, archive, index, and retrieve under a context budget [9]. xMemory decouples interaction histories into memory components and organizes them hierarchically for retrieval [4]. InfMem trains retrieve, write, and stop decisions for bounded-memory long-context reasoning [8]. MAPLE explicitly separates memory, learning, and personalization in user-adaptive agents [6]. ParamMem encodes reflective patterns into parameters, while MIA combines parametric and non-parametric memory with test-time evolution [7, 12]. MemCollab constructs shared agent-agnostic memory across heterogeneous agents [1]. SkillRL distills experience into hierarchical skill libraries [10]. MemoryArena argues that memory evaluation must test how memory guides later actions in interdependent multi-session tasks, not only static recall [3].

These papers do not claim to solve personal data governance. Many are technical contributions aimed at performance, efficiency, or agent capability. Their governance relevance is architectural. They show that personal memory can stop looking like a file, record, or conversation log. It can become an adaptive pipeline of selection, compression, retrieval, consolidation, transfer, and behavioral conditioning. If personal agents are evaluated only by outcome quality,

then the evaluation may reward architectures that personalize well while leaving memory difficult to inspect.

This paper makes three contributions. First, it identifies a personal memory audit gap in user-adaptive agent evaluation. Second, it offers a taxonomy of six memory transformations that make auditability harder. Third, it proposes the Personal Memory Audit Protocol, a practical evaluation layer for personal agents and personalization benchmarks.

2 Why Personal Memory Is Becoming Harder to Audit

The simplest memory design is a record store: interaction data enters a database; retrieval reads from that database; deletion removes a record. Real personal agents are moving away from that model. Table 1 summarizes the relevant shift.

The first transformation is *design search*. ALMA starts from the observation that most existing memory designs are human-crafted and fixed. It introduces a meta-agent that searches over memory designs expressed as executable code, including database schemas and retrieval and update mechanisms [11]. This is attractive for personal intelligence because different domains may need different memory policies. A conversational companion, scheduling assistant, and long-horizon research agent do not need the same memory design. The audit problem is that the policy a user depends on may be generated or selected by an optimization process. The relevant evaluation question is therefore not only whether the learned memory improves performance, but whether the final memory behavior can be inspected and constrained.

The second transformation is *indexed archiving*. Memex responds to the context-window bottleneck by keeping concise structured summaries and stable indices in the working context while storing full-fidelity interactions externally. The agent can later dereference an index to recover exact evidence; MemexRL optimizes what to summarize, archive, index, and retrieve [9]. This architecture is audit-friendly in one respect: stable indices can point to exact archived content. But it is not automatically audit-complete. A personal agent evaluator must still ask whether every user-derived fact in an answer can be traced back to a stable index and whether the agent can explain why it dereferenced that index.

The third transformation is *decoupling and aggregation*. xMemory argues that agent memory is not a normal RAG corpus because interaction histories are bounded, coherent, and highly redundant. It first isolates reusable facts, updates, and distinguishing details, then aggregates them into a revisable hierarchy from messages to segments, memory components, and groups [4]. This is technically sensible for personal agents because users repeat themselves, revise preferences, and provide near-duplicate context. Yet the same decomposition complicates user rights and evaluation. If a user's sentence is split into multiple memory components and aggregated with similar components from later sessions, a simple "delete that conversation" operation may not remove all behavioral effects unless lineage is retained.

The fourth transformation is *control-centric memory update*. InfMem uses PRETHINK, RETRIEVE, and WRITE steps to monitor evidence

sufficiency, perform targeted retrieval, and update bounded memory. Its training recipe aligns retrieval, writing, and stopping decisions with end-task correctness; the paper reports absolute accuracy improvements of +10.17, +11.84, and +8.23 points across three backbones, with 3.9x average inference speedup through adaptive early stopping [8]. For personal intelligence, this illustrates a broader pattern: the agent increasingly controls what becomes salient. Auditability therefore requires traces of memory control decisions, not only final memory contents.

The fifth transformation is *personalization pipeline separation*. MAPLE is especially relevant to PILA because it treats memory, learning, and personalization as distinct mechanisms rather than one capability. Memory stores and retrieves; learning extracts intelligence from accumulated interactions; personalization applies that learned knowledge at response time. The paper reports a 14.6 percent improvement in personalization score and trait incorporation rising from 45 percent to 75 percent on its benchmark [6]. The implication is that user-derived information can influence behavior without being directly retrieved. A memory audit must therefore include learned user profiles and personalization application, not only episodic storage.

The sixth transformation is *consolidation and sharing*. ParamMem encodes cross-sample reflection patterns into model parameters and integrates parametric memory with episodic and cross-sample memory [12]. MemCollab constructs agent-agnostic memory through contrastive trajectory distillation and shows that collaboratively constructed memory can serve as a shared reasoning resource across diverse agents [1]. SkillRL distills raw experience into a hierarchical skill library that co-evolves with the agent policy during reinforcement learning [10]. MIA uses non-parametric memory, parametric memory agents, test-time learning, and bidirectional conversion between parametric and non-parametric memories [7]. These systems push memory toward reusable capabilities. That is valuable, but it raises a question standard personalization scores do not answer: when does user-specific experience become system-level competence, and can that transformation be audited?

3 The Personal Memory Audit Gap

We define the personal memory audit gap as the difference between what a user-adaptive system can do because of user-derived information and what a user or evaluator can reconstruct about that influence.

The gap has three layers. The first is a *lineage gap*: the evaluator cannot trace an output or behavioral adjustment back to the user interactions that produced it. The second is a *representation gap*: the original information has been transformed into summaries, components, embeddings, profiles, skills, or parameters whose relationship to the original user input is not explicit. The third is an *effect gap*: the evaluator cannot test whether correction, deletion, or constraint changes future behavior.

These gaps matter for user trust, deployment, and regulation. Data protection frameworks such as the GDPR contain principles of lawful, fair, and transparent processing, as well as rights of access, rectification, and erasure [2]. This paper does not claim that every memory transformation violates those rules. The narrower point is technical: if user-derived memory cannot be traced or tested,

Table 1: Memory transformations relevant to personal agent auditability.

Transformation	Technical pattern	Auditability risk
Design search	ALMA searches over executable memory designs, including retrieval and update mechanisms [11].	The memory policy may be discovered by optimization rather than specified as a stable human-readable rule.
Indexed archiving	Memex keeps stable indices in context while storing full evidence externally and training read/write behavior [9].	Audit quality depends on whether indices remain linked to exact archived content and retrieval decisions.
Decoupling and aggregation	xMemory decomposes messages into segments, components, and groups before retrieval [4].	A user statement may be split across memory components, making source lineage and deletion harder to verify.
Control-centric update	InfMem actively decides when evidence is sufficient, what to retrieve, and when to overwrite bounded memory [8].	Retention depends on learned control decisions, not only static storage rules.
Personalization pipeline	MAPLE decomposes memory, learning, and personalization into distinct sub-agents [6].	User-derived information may affect behavior through learned profiles, not just direct recall.
Consolidation and sharing	ParamMem, MemCollab, SkillRL, and MIA convert experience into parametric memory, shared memory, skills, or evolving planners [1, 7, 10, 12].	Memory may propagate beyond the user’s session into reusable capabilities or cross-agent resources.

then it becomes difficult to evaluate whether access, correction, or erasure has practical meaning in a personal agent.

The same gap matters even outside legal compliance. A user may ask a personal agent to stop remembering a sensitive relationship, medical condition, political preference, or location pattern. A benchmark may show that the agent personalizes well. But the user-relevant question is whether the system can demonstrate that the sensitive memory no longer affects ranking, recommendations, drafting, planning, or tool use. That is an evaluation question.

4 Personal Memory Audit Protocol

The Personal Memory Audit Protocol (PMAP) is a reporting and stress-testing layer for user-adaptive agents. It is designed to be added to existing personalization benchmarks and product evaluations. It does not require a new task corpus. Instead, it changes what must be reported when an agent uses memory.

PMAP contains six audit questions.

Q1: Source lineage. Can the system trace each user-derived memory item, profile attribute, or action-relevant preference to the interaction or document from which it came? For indexed systems, this means stable links from outputs to archived content. For decomposed systems, this means lineage from memory components back to raw messages or sessions.

Q2: Transformation traceability. Can the system report how raw user information was transformed? Transformations may include summarization, embedding, clustering, decoupling, profile update, skill extraction, parametric consolidation, or sharing across agents. The audit does not require exposing proprietary weights. It requires enough metadata to explain which type of transformation occurred.

Q3: Retrieval accountability. Can the system show which memories were considered, selected, ignored, or dereferenced for a given response or action? This is especially important when the

agent has learned read/write/stop policies, as in systems that optimize memory control decisions [8, 9].

Q4: Correction and deletion effectiveness. After a user corrects or deletes a memory item, does future behavior change accordingly? The test should include exact recall, paraphrased recall, recommendation, planning, and tool-use behavior. A deletion mechanism that only removes a visible record but leaves profile, skill, or parametric effects untouched should fail this dimension.

Q5: Cross-agent propagation. If memory can be shared across agents, models, devices, or workflows, can the system report where the memory or derived abstraction propagated? This question is motivated by shared and collaborative memory systems such as MemCollab [1].

Q6: User-comprehensible inspection. Can a non-expert user inspect memory in a form that supports practical action? A raw vector, hidden prompt, or opaque profile score may be useful to an engineer but not to a user. PMAP therefore distinguishes developer-facing traceability from user-facing memory inspection.

Table 2 lists the minimum artifacts. These artifacts should be evaluated at two levels. The *developer level* asks whether the system can produce a faithful technical trace. The *user level* asks whether the trace can be presented in a way that supports user understanding and control. A system can pass the developer level while failing the user level.

5 Stress Tests for User-Adaptive Agents

PMAP becomes concrete through stress tests. We propose five test families.

Preference correction. The user first expresses a preference, later contradicts it, and then asks for a recommendation. The evaluator checks whether the system updates the preference, preserves the correction lineage, and avoids using the stale memory.

Table 2: PMAP audit artifacts.

Artifact	Minimum content
Memory inventory	User-visible list of active remembered facts, preferences, constraints, and inferred profile attributes.
Lineage map	Links from each memory item or profile attribute to source interaction, document, session, or task.
Transformation log	Record of summarization, aggregation, decoupling, embedding, skill extraction, sharing, or consolidation.
Retrieval trace	Memories considered and used for a response or action, including dereferenced indices where applicable.
Correction test	Before-and-after behavior showing whether correction changes retrieval, profile, and output behavior.
Deletion test	Before-and-after behavior showing whether deletion removes recall and downstream behavioral influence.

Sensitive deletion. The user provides sensitive information, uses the agent for several sessions, deletes the information, and later asks a task where that information would be useful. The evaluator checks explicit recall and indirect behavioral influence.

Context collision. Two users or personas share a device, project, or agent workspace. The evaluator checks whether memories are isolated correctly and whether one user’s preferences influence another user’s outputs.

Derived skill contamination. The agent learns a reusable heuristic or skill from one user’s workflow. The evaluator tests whether that skill encodes user-specific information and whether it can be separated from general competence.

Cross-agent transfer. A memory or derived abstraction is made available to another agent or model. The evaluator checks whether propagation is logged and whether correction or deletion flows across the receiving systems.

These tests align with MemoryArena’s insight that memory should be evaluated in loops where earlier interactions affect later actions [3]. Static recall is insufficient. A personal agent may correctly recall a preference and still fail auditability if that preference cannot be corrected, traced, or constrained across future actions.

6 Reporting Template

For PILA-style evaluations, we recommend adding a memory audit card alongside task metrics. The card should report:

- (1) the memory architecture category, including whether memory is episodic, semantic, profile-based, skill-based, shared, or parametric;
- (2) the user-derived data types stored or inferred;
- (3) the write policy, including whether it is rule-based, model-generated, learned, or user-confirmed;
- (4) the retrieval policy, including whether retrieval is top-k, hierarchical, index-dereferenced, controller-driven, or task-aware;

- (5) the transformation pipeline from raw input to memory representation;
- (6) the user inspection interface;
- (7) correction and deletion tests, including behavioral outcomes;
- (8) any cross-agent, cross-session, or parametric consolidation path.

This template is deliberately modest. It does not require revealing all proprietary implementation details. It asks for the information needed to evaluate whether personal memory remains accountable to the person it models.

7 Limitations

This paper is a conceptual evaluation-framework contribution. It does not introduce a new benchmark, dataset, or user study. Its claim is therefore limited: memory auditability should be measured when evaluating personal agents, and current agent-memory trends make that measurement increasingly important. Future work should operationalize PMAP into benchmark tasks, run user studies on memory inspection interfaces, and test whether auditability requirements reduce personalization performance or can be implemented with low overhead.

The protocol also does not solve every privacy problem. A system can be auditable and still collect too much data. Conversely, a privacy-preserving system may intentionally minimize traces, making audit design harder. The goal is not maximal logging. The goal is accountable logging: enough traceability to support user control, evaluator testing, and meaningful correction or deletion.

8 Conclusion

Personal intelligence will be built on memory, but memory is not a neutral implementation detail. It determines how an agent learns a user, how it adapts over time, and whether users can understand or contest that adaptation. Recent agent-memory research shows a clear movement toward learned, hierarchical, controller-driven, shared, and parametric memory systems. These architectures can improve personal agents, but they also make memory harder to inspect.

PILA evaluations should therefore treat memory auditability as a core property of user-adaptive agents. A personal agent should not only perform better because it remembers. It should be able to show what it remembers, where the memory came from, how it was transformed, when it was used, where it propagated, and whether correction or deletion changes behavior. Without that audit layer, personal intelligence risks becoming personalized opacity. With it, personalization can become more trustworthy, controllable, and genuinely user-centered.

References

- [1] Yurui Chang, Yiran Wu, Qingyun Wu, and Lu Lin. 2026. MemCollab: Cross-Agent Memory Collaboration via Contrastive Trajectory Distillation. *arXiv preprint arXiv:2603.23234* (2026). <https://arxiv.org/abs/2603.23234>
- [2] European Union. 2016. Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data. Official Journal of the European Union, L119, 1–88. <https://eur-lex.europa.eu/eli/reg/2016/679/oj> General Data Protection Regulation.
- [3] Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley,

- Yejin Choi, and Alex Pentland. 2026. Benchmarking Agent Memory in Interdependent Multi-Session Agentic Tasks. *arXiv preprint arXiv:2602.16313* (2026). <https://arxiv.org/abs/2602.16313>
- [4] Zhanghao Hu, Qinglin Zhu, Runcong Zhao, Di Liang, Hanqi Yan, Yulan He, and Lin Gui. 2026. xMemory: Beyond RAG for Agent Memory: Retrieval by Decoupling and Aggregation. *arXiv preprint arXiv:2602.02007* (2026). <https://arxiv.org/abs/2602.02007>
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. 9459–9474. <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [6] Deepak Babu Piskala. 2026. MAPLE: A Sub-Agent Architecture for Memory, Learning, and Personalization in Agentic AI Systems. *arXiv preprint arXiv:2602.13258* (2026). <https://arxiv.org/abs/2602.13258>
- [7] Jingyang Qiao, Weicheng Meng, Yu Cheng, Zhihang Lin, Zhizhong Zhang, Xin Tan, Jingyu Gong, Kun Shao, and Yuan Xie. 2026. Memory Intelligence Agent. *arXiv preprint arXiv:2604.04503* (2026). <https://arxiv.org/abs/2604.04503>
- [8] Xinyu Wang, Mingze Li, Peng Lu, Xiao-Wen Chang, Lifeng Shang, Jinpeng Li, Fei Mi, Prasanna Parthasarathi, and Yufei Cui. 2026. InfMem: Learning System-2 Memory Control for Long-Context Agent. *arXiv preprint arXiv:2602.02704* (2026). <https://arxiv.org/abs/2602.02704>
- [9] Zhenfeng Wang, Huancheng Chen, Jiayun Wang, and Wei Wei. 2026. Memex(RL): Scaling Long-Horizon LLM Agents via Indexed Experience Memory. *arXiv preprint arXiv:2603.04257* (2026). <https://arxiv.org/abs/2603.04257>
- [10] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. *arXiv preprint arXiv:2602.08234* (2026). <https://arxiv.org/abs/2602.08234>
- [11] Yiming Xiong, Shengran Hu, and Jeff Clune. 2026. Learning to Continually Learn via Meta-learning Agentic Memory Designs. *arXiv preprint arXiv:2602.07755* (2026). <https://arxiv.org/abs/2602.07755>
- [12] Tianjun Yao, Yongqiang Chen, Yujia Zheng, Pan Li, Zhiqiang Shen, and Kun Zhang. 2026. ParamMem: Augmenting Language Agents with Parametric Reflective Memory. *arXiv preprint arXiv:2602.23320* (2026). <https://arxiv.org/abs/2602.23320>